

先把相同区间合并，按严格包含关系建出依赖树，原来的每次操作仍按输入顺序映射到对应结点。再加一个表示整个序列的虚根。

以下讨论一个结点的状态时，只使用它和它的后代对应的操作，不借助祖先覆盖。一个结点被自身操作成功覆盖后，区间一定是纯色，之后属于它或它后代的操作都无法再改变这段颜色。若始终不让这个结点成功覆盖，各个儿子互不相交，可以独立选择方案。因此一个结点的可达状态，除了所有儿子状态的积，只可能多出若干种纯色。先求这些纯色。

固定颜色 c ，若结点 v 已经能变成纯色 c ，就称 v 已激活。设 mx_v 为当前在 I_v 中最多能得到多少个 c ， $gapCnt_v[c]$ 为 I_v 中没有落入任何儿子区间的 c 的数量。未激活时有

$$mx_v = gapCnt_v[c] + \sum_{u \in son_v} mx_u.$$

儿子之间互不影响，所以右侧的最大值可以同时取得。于是 v 当前的操作能使它激活，当且仅当

$$2mx_v > |I_v|.$$

记 $cnt_v[c]$ 为初始序列在 I_v 中的 c 的数量，初始时 $mx_v = cnt_v[c]$ 。若 v 激活前的最大值为 cur ，激活后它变成 $|I_v|$ ，增量为

$$\Delta = |I_v| - cur.$$

这份增量只会传给 v 的未激活祖先，直到最近的已激活祖先 r 为止，因为 r 及其祖先已经把 r 的贡献视为 $|I_r|$ 。

对树求 Euler 序，在 tin_v 加上 Δ ；若 r 存在，再在 tin_r 减去 Δ 。于是任意未激活结点 x 当前的最大值都是

$$mx_x = cnt_x[c] + \text{sum}(tin_x, tout_x),$$

其中子树和用树状数组维护。激活 v 时，再给整个 $subtree_v$ 加上标记 (dep_v, v) 。线段树支持区间加入标记、单点取最大值，就能找到某个结点最近的已激活祖先。

按时间顺序扫描所有操作即可求出颜色 c 能激活哪些结点。这里只分别判断每种纯色是否可达，不要求它们由同一个方案得到，因此各颜色可以独立处理。记结点 v 最终得到的颜色集合为 $good_v$ 。

最后自底向上计数。设 $ways_v$ 为 I_v 内不同可达状态数， uni_v 为其中的纯色集合。先考虑没有被 v 自身操作成功覆盖的状态：

$$base_v = \prod_{u \in son_v} ways_u.$$

令 $mask_v$ 表示这些状态中可以得到哪些纯色。若儿子以外的部分为空，则它对所有颜色都没有限制；否则只有这部分原本全为同一种颜色时，该颜色才可能出现。再将这个集合与所有儿子的 uni 取交即可。

$good_v$ 中每种颜色各贡献一个纯色状态，其中已经属于 $mask_v$ 的不能重复计算。因此

$$ways_v = base_v + |good_v \setminus mask_v|,$$

$$uni_v = mask_v \cup good_v.$$

所有计数对 998244353 取模。 $ways_0$ 即为答案。

时间复杂度为 $O(m \log m + K(n + m) \log(n + m))$ 。